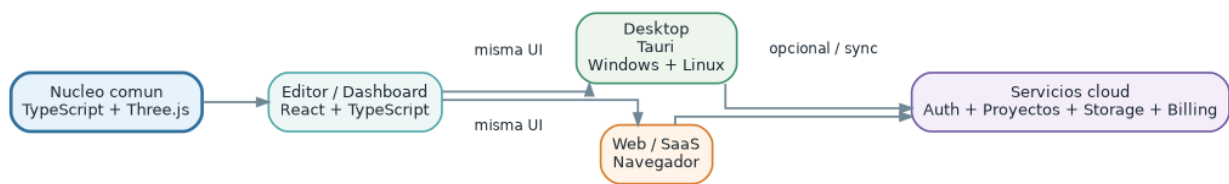


# 3D Asset Platform

## Product, Architecture and Development Roadmap

A modular JavaScript/TypeScript platform for creating professional 3D game assets

**Core decision.** Build one shared 3D editor and domain core from day zero. Deliver the first commercial product as a Windows/Linux desktop application, while keeping the same frontend and core deployable as a browser SaaS. Do not create a desktop-only codebase that later has to be rewritten for the web.



|                             |              |                                                                                                                                                                      |
|-----------------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Document purpose</b>     |              | Give the programmer an implementation-ready plan: product scope, architecture, modules, contracts, UI, export pipeline, milestones, testing and acceptance criteria. |
| <b>Working product name</b> |              | 3D Asset Forge (provisional; branding can change without affecting architecture).                                                                                    |
| <b>Primary audience</b>     | <b>audi-</b> | Technical lead, frontend/3D engineer, backend engineer, product owner.                                                                                               |
| <b>Primary forms</b>        | <b>plat-</b> | Windows and Linux desktop first; web/SaaS as a first-class target of the same codebase; macOS possible later.                                                        |
| <b>Core technology</b>      |              | TypeScript/Javascript, Three.js, React, Tauri, modular backend when SaaS features are enabled.                                                                       |

# Contents

|                                                               |           |
|---------------------------------------------------------------|-----------|
| <b>1 Executive decision</b>                                   | <b>2</b>  |
| 1.1 What we are actually building . . . . .                   | 2         |
| 1.2 Desktop, SaaS, or both? . . . . .                         | 2         |
| <b>2 Product principles</b>                                   | <b>2</b>  |
| <b>3 User experience: a simpler professional 3D workbench</b> | <b>3</b>  |
| 3.1 Main dashboard / editor shell . . . . .                   | 3         |
| 3.2 Required zones . . . . .                                  | 3         |
| 3.3 Interaction goal . . . . .                                | 4         |
| <b>4 Technical architecture</b>                               | <b>4</b>  |
| 4.1 Clean Architecture plus feature modularity . . . . .      | 4         |
| 4.1.1 Domain layer . . . . .                                  | 5         |
| 4.1.2 Application layer . . . . .                             | 5         |
| 4.1.3 Adapters / infrastructure . . . . .                     | 5         |
| 4.1.4 Presentation layer . . . . .                            | 6         |
| <b>5 Module model</b>                                         | <b>7</b>  |
| 5.1 Core modules for the first product . . . . .              | 8         |
| <b>6 How modules affect the dashboard without coupling</b>    | <b>8</b>  |
| <b>7 Recommended repository structure</b>                     | <b>9</b>  |
| 7.1 Dependency constraints . . . . .                          | 10        |
| <b>8 Technology baseline</b>                                  | <b>10</b> |
| 8.1 Why Three.js stays behind an adapter . . . . .            | 10        |
| <b>9 3D editor capability roadmap</b>                         | <b>11</b> |
| 9.1 Minimum creation tools . . . . .                          | 11        |
| 9.2 Tools deliberately postponed . . . . .                    | 11        |
| <b>10 Procedural generator system</b>                         | <b>11</b> |
| 10.1 Generator contract . . . . .                             | 11        |
| 10.2 First generator families . . . . .                       | 12        |
| 10.3 Randomize without chaos . . . . .                        | 12        |
| <b>11 Project and file format</b>                             | <b>12</b> |
| 11.1 Project package . . . . .                                | 12        |
| 11.2 Essential fields . . . . .                               | 13        |
| 11.3 Migration rule . . . . .                                 | 13        |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| <b>12</b> | <b>Export strategy</b>                                | <b>13</b> |
| 12.1      | Primary format: GLB/glTF . . . . .                    | 13        |
| 12.2      | Secondary formats . . . . .                           | 14        |
| 12.3      | Engine presets . . . . .                              | 14        |
| 12.4      | Export acceptance . . . . .                           | 14        |
| <b>13</b> | <b>Desktop application architecture</b>               | <b>14</b> |
| 13.1      | Why release desktop first . . . . .                   | 14        |
| 13.2      | Desktop host responsibilities . . . . .               | 15        |
| <b>14</b> | <b>SaaS architecture</b>                              | <b>15</b> |
| 14.1      | What SaaS adds . . . . .                              | 15        |
| 14.2      | Cloud components . . . . .                            | 15        |
| <b>15</b> | <b>AI roadmap</b>                                     | <b>16</b> |
| 15.1      | AI Phase 1: prompt to parameters . . . . .            | 16        |
| 15.2      | AI Phase 2 and later . . . . .                        | 16        |
| <b>16</b> | <b>Plugin architecture</b>                            | <b>16</b> |
| 16.1      | Plugin contribution points . . . . .                  | 17        |
| 16.2      | Plugin manifest . . . . .                             | 17        |
| 16.3      | Isolation policy . . . . .                            | 17        |
| <b>17</b> | <b>Security and resilience</b>                        | <b>17</b> |
| <b>18</b> | <b>Testing strategy</b>                               | <b>18</b> |
| 18.1      | Test pyramid . . . . .                                | 18        |
| 18.2      | Non-negotiable deterministic tests . . . . .          | 18        |
| <b>19</b> | <b>Observability and developer diagnostics</b>        | <b>18</b> |
| <b>20</b> | <b>Roadmap: short, medium and long term</b>           | <b>19</b> |
| 20.1      | Phase 0 - Foundation . . . . .                        | 19        |
| 20.2      | Phase 1 - Fast visible vertical slice . . . . .       | 19        |
| 20.3      | Phase 2 - Editor Core . . . . .                       | 20        |
| 20.4      | Phase 3 - Procedural creation . . . . .               | 20        |
| 20.5      | Phase 4 - Professional export center . . . . .        | 20        |
| 20.6      | Phase 5 - Desktop Alpha for Windows/Linux . . . . .   | 20        |
| 20.7      | Phase 6 - SaaS foundation . . . . .                   | 21        |
| 20.8      | Phase 7 - AI, marketplace and collaboration . . . . . | 21        |
| <b>21</b> | <b>Suggested planning horizon</b>                     | <b>22</b> |
| <b>22</b> | <b>First implementation backlog</b>                   | <b>22</b> |

|    |                                                       |    |
|----|-------------------------------------------------------|----|
| 23 | Acceptance criteria for the first public-looking demo | 23 |
| 24 | What not to build yet                                 | 23 |
| 25 | Definition of architectural success                   | 23 |
| 26 | Final architecture mandate for the programmer         | 24 |
|    | References and technical basis                        | 25 |

# 1 Executive decision

## 1.1 What we are actually building

The product is not a collection of hard-coded 3D objects and it is not a reduced clone of Blender. It is a focused creation environment for game-ready 3D assets whose main advantage is simplicity, procedural generation and fast export.

The user should be able to open the application, create an object, modify it visually, generate variants, validate it and export it without understanding low-level 3D APIs. The platform hides technical complexity while still producing professional files.

**Long-term product sentence:** “A simple professional workbench that lets game developers create, customize, validate and export 3D assets quickly, using direct controls, procedural generators and later AI assistance.”

## 1.2 Desktop, SaaS, or both?

**Recommendation: both, with a desktop-first commercial release and a dual-target architecture from the beginning.**

This distinction is crucial:

- **Desktop-first release** means the first complete product should run locally on Windows and Linux. This reduces early cloud cost, handles large local files naturally, works offline and gives direct filesystem access.
- **Web-first technical foundation** means the editor UI and 3D engine are built with browser technologies and remain deployable in a normal browser.
- **SaaS later** adds accounts, cloud projects, storage, collaboration, credits, billing, AI jobs and marketplace services around the same editor.

Tauri is suitable for this model because it packages HTML/JavaScript/CSS frontends as desktop applications while providing native capabilities through its host layer [R1-R3]. The result is one product core, not two separate products.

| Option                 | Advantage                                      | Main problem                                           | Decision               |
|------------------------|------------------------------------------------|--------------------------------------------------------|------------------------|
| Desktop only           | Fast local iteration; offline; file access     | Later SaaS rewrite risk; no cloud business model       | Reject as architecture |
| SaaS only from day one | Easy subscriptions; zero install               | Early backend complexity; uploads; cloud cost; latency | Not first release      |
| Hybrid shared core     | One editor; desktop and browser; gradual cloud | Requires strict separation of local/cloud adapters     | <b>Chosen</b>          |

# 2 Product principles

The following rules are architectural constraints, not optional preferences.

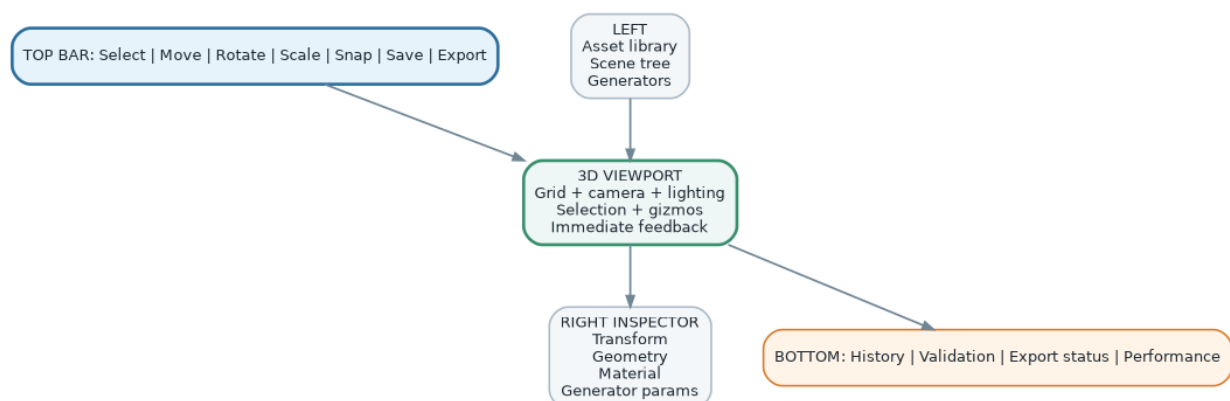
1. **Visible progress first.** Every development phase must end with something the user can see and operate in the central editor.

2. **One central workbench.** The main dashboard is the interaction surface for all feature modules.
3. **Modules do not own the whole application.** A generator, exporter or AI provider cannot directly manipulate unrelated UI or persistence.
4. **Dependency inversion.** Domain and application logic never depend on Three.js, React, Tauri, a cloud vendor or an AI vendor.
5. **Replaceable adapters.** Local filesystem, cloud storage, export backends and AI providers are accessed through ports/contracts.
6. **No early microservices.** Start as a modular monolith. Split only when operational evidence requires it.
7. **Project format is a product asset.** A project must be serializable, versioned, migratable and recoverable.
8. **Undo/redo is structural.** Every destructive editor operation must be represented as an explicit command.
9. **Export is validated.** Generating a file is not success; the file must pass structural checks.
10. **Professional defaults, simple interface.** Complexity is progressively revealed rather than placed on the first screen.

### 3 User experience: a simpler professional 3D workbench

#### 3.1 Main dashboard / editor shell

The central screen must behave as the “display” of all modules. Modules contribute tools, panels, status indicators and actions to the shell, but they remain independently implemented.



**Figure 1:** Proposed main workbench. The 3D viewport remains central while modules surface controls around it.

#### 3.2 Required zones

**Top toolbar** Select, Move, Rotate, Scale, local/world coordinates, snapping, duplicate, group, save, undo, redo, preview and export.

**Left panel** Scene hierarchy, primitive library, procedural generators, templates and later user assets.

**Center viewport** Real-time object view with grid, camera controls, lighting, object selection, gizmos, overlays and immediate feedback.

**Right inspector** Context-sensitive properties: transform, geometry parameters, material, generator-specific parameters, metadata and export hints.

**Bottom status area** History, validation errors/warnings, export status, selected-object statistics and performance information.

### 3.3 Interaction goal

A beginner should be able to produce a useful first asset through a short path:

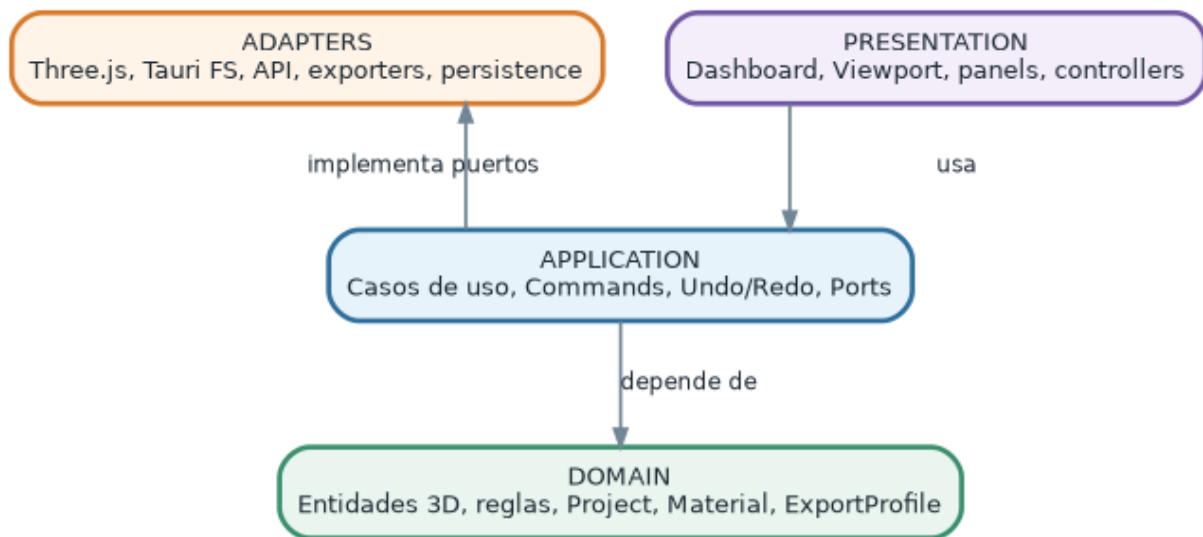
1. New Project.
2. Add Box or choose “Sci-Fi Crate Generator”.
3. Change dimensions and style using visible controls.
4. Move/rotate/scale directly in the viewport.
5. Choose a material.
6. Press Validate.
7. Press Export GLB.

No scripting is required for this basic flow. Scripting and advanced procedural graphs are later expert features.

## 4 Technical architecture

### 4.1 Clean Architecture plus feature modularity

A classic MVC label alone is not enough for a 3D editor. The recommendation is **Clean Architecture as the dependency rule**, with an MVC-like presentation organization inside each feature.



**Figure 2:** Dependency direction. Infrastructure and UI depend inward; the domain never depends on frameworks.

#### 4.1.1 Domain layer

Pure TypeScript. No React, no Three.js, no filesystem, no HTTP.

Examples:

- AssetDocument, SceneNode, Transform, GeometryDefinition
- MaterialDefinition, TextureReference, GeneratorDefinition
- ExportProfile, ValidationIssue, ProjectMetadata
- value objects for IDs, units, version numbers and coordinate conventions

#### 4.1.2 Application layer

Contains use cases and coordination:

- CreateProject, AddPrimitive, SelectObject, ApplyTransform
- ChangeMaterial, RunGenerator, DuplicateObject
- ExecuteCommand, UndoCommand, RedoCommand
- SaveProject, LoadProject, ValidateProject, ExportAsset

It defines ports such as ProjectRepository, GeometryRenderer, AssetExporter, FileSystemPort, CloudStoragePort, AiGenerationPort.

#### 4.1.3 Adapters / infrastructure

Concrete technology lives here:

- Three.js renderer and geometry adapter
- Tauri local filesystem adapter
- browser IndexedDB adapter



- REST/WebSocket cloud API adapter
- GLB/glTF/OBJ/STL exporters
- SaaS storage, database, auth and billing adapters
- AI provider adapters

4.1.4 Presentation layer

React/TypeScript views and controllers. The view never performs 3D mutations directly. It sends a command/use-case request and renders the resulting read model.

**Strict rule for the programmer:** React components must not import business entities from infrastructure, and feature modules must not directly import another feature’s internal classes. Inter-feature communication happens through public contracts, commands, events or shared domain types.

5 Module model

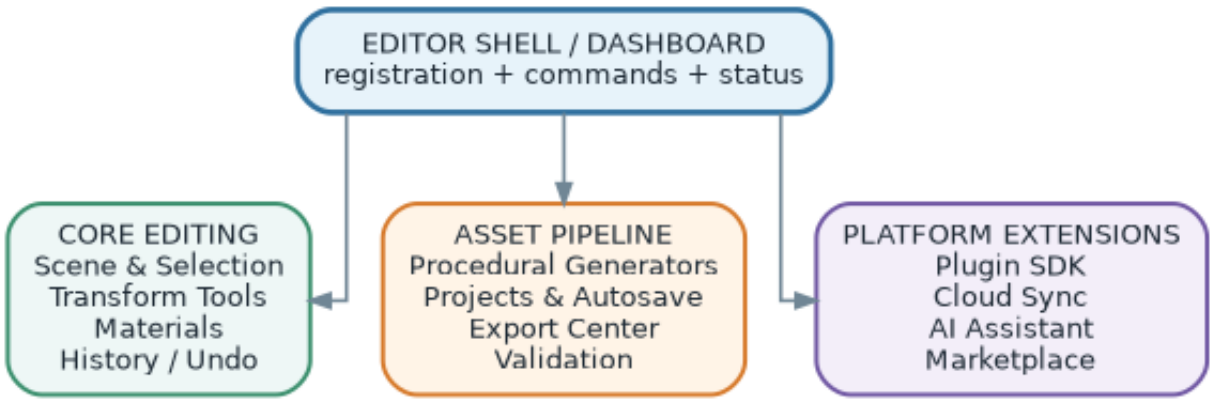


Figure 3: Feature modules attached to the editor shell through explicit contracts.

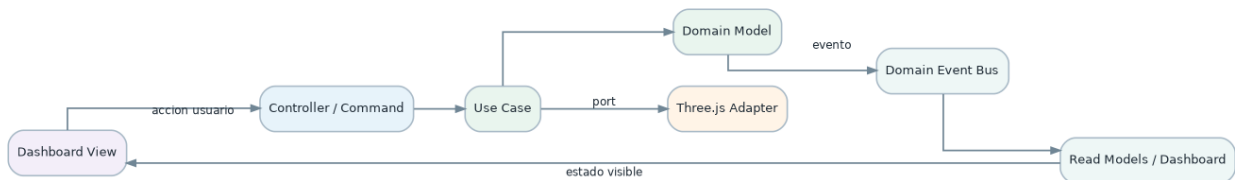
5.1 Core modules for the first product

| Module          | Responsibility                                                                      | Must not do                             |
|-----------------|-------------------------------------------------------------------------------------|-----------------------------------------|
| Editor Shell    | Layout, tool registration, panel registration, global keyboard routing, status area | Implement geometry/export logic         |
| Viewport        | Camera, render loop, picking, overlays, gizmos                                      | Persist projects or call cloud directly |
| Scene           | Scene graph, hierarchy, selection, grouping, visibility                             | Know about UI components                |
| Transform Tools | Move, rotate, scale, snapping, pivot operations                                     | Own renderer state outside its contract |
| Geometry        | Primitive definitions and geometry operations                                       | Know about Tauri or SaaS                |

| Module        | Responsibility                                              | Must not do                               |
|---------------|-------------------------------------------------------------|-------------------------------------------|
| Materials     | PBR material model, presets, textures                       | Export files directly                     |
| Generators    | Procedural object families and parameters                   | Bypass command/history system             |
| History       | Commands, undo/redo, transactional edits                    | Contain UI-specific logic                 |
| Projects      | New, open, save, autosave, migrations, recovery             | Depend on a single persistence technology |
| Validation    | Asset/project checks and diagnostics                        | Modify assets silently                    |
| Export Center | Export profiles, job coordination, result report            | Encode all exporters in one giant class   |
| Plugin SDK    | Module manifests, registrations, permissions, compatibility | Expose unrestricted internal state        |
| Cloud Sync    | Account-aware upload/download/version sync                  | Be required for offline desktop editing   |
| AI Assistant  | Prompt interpretation and generation planning               | Directly mutate scene without a command   |
| Marketplace   | Catalog, licensing metadata, purchases later                | Be embedded into core domain rules        |

## 6 How modules affect the dashboard without coupling

The central dashboard should reflect actions from every module. This is accomplished through commands and domain events rather than direct cross-module calls.



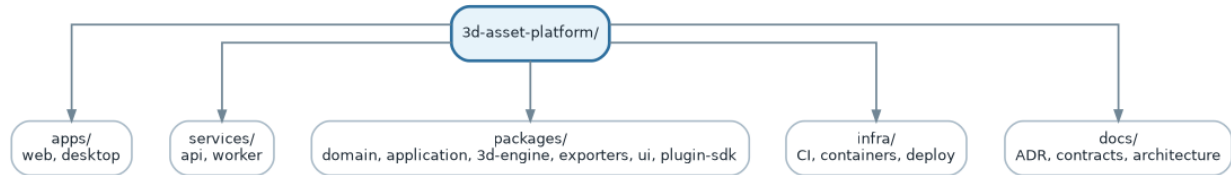
**Figure 4:** Example data flow from a user action to the domain and back to visible dashboard state.

Example: the user changes a crate width from 1.0 m to 1.5 m.

1. Inspector emits a `ChangeGeneratorParameterCommand`.
2. Application validates the parameter and applies it to the document model.
3. Generator service rebuilds the geometry through a port.
4. Domain publishes `AssetGeometryChanged`.
5. Viewport projection rerenders the object.
6. History records the command for Undo.
7. Validation invalidates its previous report.
8. Dashboard status updates polygon count and dirty-state indicator.

No inspector code knows how GLB export works. No exporter code knows how React renders a slider.

## 7 Recommended repository structure



**Figure 5:** Monorepo organization: applications and infrastructure around shared packages.

```

3d-asset-platform/
  apps/
    web/           # browser editor / future SaaS frontend
    desktop/       # Tauri host; reuses web/editor packages
  services/
    api/           # SaaS API, only required for cloud features
    worker/        # later: conversion, AI, optimization jobs
  packages/
    domain/        # pure business/domain model
    application/   # use cases, commands, ports
    3d-engine/     # Three.js implementation behind interfaces
    editor-shell/  # shared workbench orchestration
    scene/         # scene feature
    materials/     # material feature
    generators/    # procedural generator registry
    exporters/     # exporter registry
    project-format/ # schemas, migration, serialization
    validation/    # quality rules
    plugin-sdk/    # extension contracts
    ui/            # reusable design system/components
    shared/        # small cross-cutting primitives only
    testing/       # fixtures and test helpers
  infra/
    ci/
    containers/
    deploy/
  docs/
    architecture/
    adr/           # Architecture Decision Records
    contracts/
  
```

### 7.1 Dependency constraints

Recommended package direction:

```
domain <- application <- feature modules/adapters <- apps
```

Automated lint rules should forbid imports that violate this direction. Circular dependencies fail CI.

## 8 Technology baseline

| Area              |         | Recommendation                                                    | Reason                                                                                       |
|-------------------|---------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Language          |         | TypeScript                                                        | Safer contracts and refactoring than untyped JavaScript while remaining JavaScript ecosystem |
| UI                |         | React + TypeScript                                                | Mature component model for complex editor panels                                             |
| 3D rendering      |         | Three.js                                                          | Browser-native 3D engine with official exporters and transformation utilities [R4-R6]        |
| Desktop shell     |         | Tauri 2                                                           | Shared HTML/JS/CSS frontend with native desktop capabilities [R1-R3]                         |
| Build             |         | Vite-class modern bundling                                        | Fast frontend iteration; keep build adapter replaceable                                      |
| Project<br>tence  | persis- | Versioned JSON meta-<br>data + binary/texture<br>asset references | Inspectable and migratable project format                                                    |
| Desktop<br>tence  | persis- | Tauri filesystem<br>adapter                                       | Direct local project and export access                                                       |
| Browser<br>tence  | persis- | IndexedDB/local cache                                             | Offline-capable browser drafts before SaaS                                                   |
| SaaS API          |         | TypeScript modular API<br>(e.g. NestJS-style archi-<br>tecture)   | Strong module boundaries and dependency injection                                            |
| Database          |         | PostgreSQL                                                        | Projects, users, permissions, billing metadata                                               |
| Object storage    |         | S3-compatible abstrac-<br>tion                                    | Large models, textures, previews and versions                                                |
| Primary<br>change | ex-     | GLB/gITF                                                          | Open runtime 3D standard [R7-R8]                                                             |
| Validation        |         | Khronos glTF Validator                                            | Conformance report and asset statistics [R9]                                                 |

### 8.1 Why Three.js stays behind an adapter

Three.js provides the scene graph and export tooling, including GLTFExporter and other exporters [R4-R6]. However, domain objects should not be `THREE.Mesh` instances. The domain describes an asset; the adapter converts that description to renderable Three.js objects. This lets testing run without WebGL and keeps future engine replacement possible.

React Three Fiber can be considered only as a presentation/render integration layer; it is a React renderer for Three.js [R10]. It must not become the domain model.

## 9 3D editor capability roadmap

### 9.1 Minimum creation tools

The first professional editor baseline should support:

- Box, cylinder, sphere, plane
- select/deselect/multi-select
- move/rotate/scale
- world/local transform modes
- grid and snapping
- duplicate/delete
- group/ungroup
- scene hierarchy

- object visibility/locking
- named objects
- basic camera presets
- orbit/pan/zoom
- perspective/orthographic view
- basic lights and environment
- material presets
- base color, metallic, roughness
- texture assignment later in baseline
- object statistics
- undo/redo
- autosave

Three.js includes TransformControls, whose interaction model is explicitly similar to 3D DCC manipulation tools, making it suitable for the move/rotate/scale gizmo layer [R5].

## 9.2 Tools deliberately postponed

Do not attempt to implement Blender-level functionality at the start:

- sculpting
- full mesh vertex/edge/face editing
- UV unwrapping suite
- rigging and skeletal animation editor
- simulation/physics authoring
- compositor
- shader node editor

These are separate product areas. The platform wins by being faster and simpler for asset creation, not by matching Blender feature-for-feature.

# 10 Procedural generator system

This is the product differentiator.

## 10.1 Generator contract

Every procedural generator must expose a stable manifest:

```
GeneratorDefinition {  
  id: string  
  version: string  
  name: string  
  category: string  
  parameterSchema: ParameterSchema  
  defaultParameters: object  
  create(parameters, seed): AssetDescription  
  migrate(oldVersion, data): data  
}
```

## 10.2 First generator families

### 1. Sci-Fi Crate

2. Industrial Barrel
3. Container
4. Door / Airlock
5. Terminal / Console

Each generator must support deterministic seeds. Same generator version + same parameters + same seed must reproduce the same logical asset. This supports packs, undo/redo, project reopening and cloud synchronization.

### 10.3 Randomize without chaos

“Randomize” should modify constrained style parameters, not generate arbitrary broken geometry. Example groups:

- proportions
- panel density
- corner reinforcement
- vents
- handles
- emissive lights
- damage amount
- color/material preset

## 11 Project and file format

### 11.1 Project package

Use a versioned project structure such as:

```
MyAsset.3daf/  
  project.json  
  assets/  
    textures/  
    previews/  
    cache/
```

For desktop distribution this can later become a zip-like single-file container, but the logical format remains explicit.

### 11.2 Essential fields

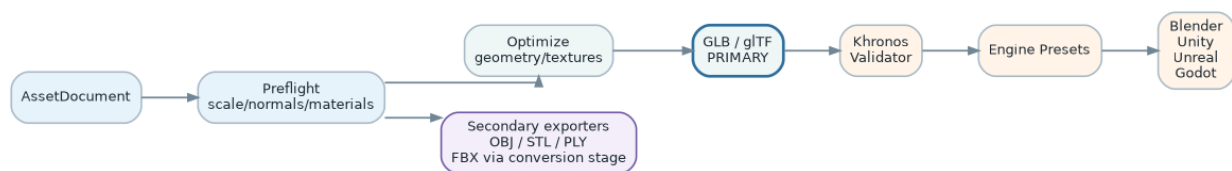
- project schema version
- application version
- units and coordinate convention
- scene nodes and hierarchy
- transforms

- geometry definitions / generator references
- materials and textures
- generator parameters and seeds
- export profiles
- validation state
- created/modified timestamps

### 11.3 Migration rule

Opening an old project must run explicit schema migrations. Never silently discard unknown fields. Before migration, create a recovery copy.

## 12 Export strategy



**Figure 6:** Export is treated as a pipeline with preflight, validation and platform presets.

### 12.1 Primary format: GLB/glTF

Khronos defines glTF as a royalty-free runtime 3D asset delivery specification designed for efficient transmission and loading [R7-R8]. Three.js provides GLTFExporter, which can emit JSON glTF or binary GLB and includes scene elements such as meshes, materials, textures and animations where supported [R4].

GLB should be the default first-class export because it is a compact single-file deliverable and provides a strong interoperability path. Blender has glTF 2.0 import/export support [R11]. Unity provides glTFast for glTF use [R12]. Unreal Engine documents glTF import [R13]. Godot supports glTF scene import and documents GLB/glTF workflows [R14].

### 12.2 Secondary formats

- **glTF folder:** for workflows that prefer separate JSON, buffers and textures.
- **OBJ:** geometry compatibility; Three.js OBJExporter does not export MTL material files, so the UI must clearly label its limitation [R6].
- **STL:** useful for geometry/printing-oriented workflows; not a game-material format.
- **PLY:** later for simple geometry/data workflows.
- **FBX:** later conversion stage, preferably through a dedicated converter/Blender integration rather than pretending Three.js has a complete direct FBX exporter.

## 12.3 Engine presets

Export presets should modify conventions without changing the underlying project:

- Unity preset
- Unreal preset
- Godot preset
- Blender interchange preset
- Generic web/glTF preset

Each preset controls scale, naming, texture packaging, coordinate transforms if needed, optional collider metadata and validation rules.

## 12.4 Export acceptance

An export is successful only if:

1. exporter completed without exception;
2. output exists and has non-zero size;
3. structural validator returns no blocking errors;
4. re-import smoke test can load the produced asset;
5. export report records warnings, statistics and profile version.

The official Khronos glTF Validator validates against glTF 2.0 and can produce JSON reports containing issues and asset statistics [R9].

# 13 Desktop application architecture

## 13.1 Why release desktop first

For the initial creation workflow, desktop gives the fastest route to a usable professional tool:

- no account required;
- no upload required;
- direct access to local textures and project files;
- offline editing;
- simpler early operations;
- easier large-asset workflows;
- export directly to game project folders.

Tauri uses a system webview architecture and supports desktop targets including Windows and Linux [R1-R3]. Its filesystem plugin gives a controlled filesystem API [R15]. Its capabilities model can restrict which windows/webviews receive native permissions [R16].

## 13.2 Desktop host responsibilities

The Tauri layer should remain thin:



- open/save dialogs
- filesystem read/write
- file watching later
- native menu/shortcuts
- application updater later
- secure credential storage adapter later
- optional local conversion worker

It should not contain the main scene model or procedural generation logic.

## 14 SaaS architecture

### 14.1 What SaaS adds

The browser editor already exists because it is the same frontend. SaaS adds services around it:

- user accounts and organization/workspace model
- cloud project storage
- project versions and backups
- device synchronization
- usage quotas/credits
- subscription/billing
- cloud procedural or AI jobs
- team sharing and permissions
- marketplace purchases and licenses
- telemetry/usage analytics with privacy controls

### 14.2 Cloud components

- API modular monolith
- PostgreSQL for relational metadata
- S3-compatible object storage for project blobs/assets/previews
- worker queue only when asynchronous jobs are actually needed
- WebSocket/realtime channel only when collaboration/sync needs it
- provider-neutral authentication port
- provider-neutral billing port

**Do not start with Kubernetes, many microservices or distributed queues.** They do not improve the editor. The initial cloud should remain a modular monolith with one deployment plus storage/database. Split services only after a module has an independent scaling, security or availability requirement.

## 15 AI roadmap

AI is a layer on top of a reliable editor, not the foundation of V1.

### 15.1 AI Phase 1: prompt to parameters

User writes: "Create a damaged black military sci-fi crate with four red lights."

AI returns a structured, validated plan:

```
{
  "generator": "scifi-crate",
  "style": "military",
  "baseColor": "black",
  "damage": 0.25,
  "emissiveLights": 4,
  "seed": 481903
}
```

The deterministic generator then builds the object. This keeps AI output inspectable and editable.

### 15.2 AI Phase 2 and later

- material suggestions
- automatic naming/tagging
- generator parameter optimization
- reference-image interpretation
- texture generation through a provider plugin
- external image-to-3D / mesh-generation integrations behind a provider port

Every AI action must create a normal editor command so the user can undo it.

## 16 Plugin architecture

A plugin system is required if the platform is intended to grow without destabilizing the core.

### 16.1 Plugin contribution points

A plugin may register only approved extension points:

- procedural generator
- tool command
- inspector panel
- asset template
- material library
- validator rule

- exporter
- import adapter
- AI provider

## 16.2 Plugin manifest

Each plugin declares ID, semantic version, compatible platform API version, requested capabilities, contribution types and optional migrations.

## 16.3 Isolation policy

Plugins do not receive arbitrary access to the entire store or filesystem. Desktop-native permissions should be mediated through capabilities/ports. This fits Tauri's capability model, which is designed to constrain access exposed to frontend webviews [R16].

# 17 Security and resilience

Even a 3D editor handles untrusted files, plugins and cloud data.

Minimum controls:

- validate imported project schemas and file types;
- size limits for textures/models;
- path traversal protection;
- no arbitrary command execution from project files;
- Content Security Policy for web/desktop webview;
- least-privilege Tauri capabilities;
- plugin capability review;
- secure secret storage for cloud tokens;
- signed desktop releases and update metadata when updater is enabled;
- project autosave, crash recovery and corruption-safe writes;
- server-side authorization independent of UI checks.

Tauri documents a capabilities system for granular frontend/native access and an official updater mechanism for desktop applications [R16-R17].

# 18 Testing strategy

## 18.1 Test pyramid

- **Domain unit tests:** transforms, project migrations, generator determinism, validation rules.
- **Application tests:** commands, undo/redo, save/load, export orchestration.
- **Adapter tests:** Three.js conversion, filesystem, IndexedDB, API and exporters.

- **Visual regression:** selected reference assets rendered from fixed cameras and compared with approved baselines.
- **End-to-end editor tests:** create -> edit -> save -> reopen -> export.
- **Cross-platform smoke tests:** Windows and Linux installers/open/save/export.

## 18.2 Non-negotiable deterministic tests

For every procedural generator:

1. fixed parameters + fixed seed -> same logical output;
2. geometry contains finite values only;
3. bounding box is valid;
4. no zero-scale nodes;
5. expected material slots exist;
6. export and re-import succeeds.

## 19 Observability and developer diagnostics

The bottom dashboard status area should expose useful development and professional diagnostics:

- FPS/frame time
- object count
- vertex/triangle count
- texture memory estimate when available
- dirty/saved project state
- latest autosave
- validation count: errors/warnings/info
- export job state
- generator ID/version/seed for selected generated asset

Production logs should be structured and categorized. Debug data must never be the only source of user-facing state.

## 20 Roadmap: short, medium and long term



**Figure 7:** Recommended delivery order. Each phase preserves a working product.

### 20.1 Phase 0 - Foundation

**Objective:** create the repository and rules that prevent future architectural collapse.

Deliver:

- monorepo skeleton;
- TypeScript strict mode;
- domain/application packages;
- app/web and app/desktop shells;
- CI with tests, lint and dependency-boundary checks;
- architecture decision records;
- module registry and event bus contract;
- base design system;
- feature flags for unfinished modules.

**Acceptance gate:** web shell and desktop shell both boot from the same editor package; a dummy feature module can register one panel without editing the shell internals.

## 20.2 Phase 1 - Fast visible vertical slice

This phase exists specifically to see results immediately.

Deliver one complete path:

1. open the editor;
2. render a cube on a grid;
3. orbit/pan/zoom camera;
4. select the cube;
5. move/rotate/scale using gizmos;
6. change dimensions and color in inspector;
7. undo/redo;
8. save a project;
9. reopen it;
10. export a GLB;
11. validate the GLB.

**Phase 1 Definition of Done:** a programmer can demonstrate the entire flow live without manually editing JSON, opening devtools or using Blender to repair the result.

## 20.3 Phase 2 - Editor Core

Deliver:

- multi-object scene;
- scene tree;
- Box/Cylinder/Sphere/Plane;
- duplicate/delete/group;

- snapping;
- material presets;
- perspective and orthographic cameras;
- object statistics;
- autosave and crash recovery;
- keyboard shortcut system;
- validation panel.

**Gate:** a basic scene can be created, saved, reopened and edited without loss.

## 20.4 Phase 3 - Procedural creation

Deliver the generator SDK and first five generator families. The generator parameter panel must be schema-driven: adding a new parameter should not require custom panel code for every field type.

**Gate:** one generator can be installed/removed without changing the editor core, and fixed seed output is reproducible.

## 20.5 Phase 4 - Professional export center

Deliver:

- GLB and glTF production exporters;
- export presets;
- OBJ and STL secondary export;
- preflight checks;
- Khronos validation integration for glTF/GLB;
- re-import smoke test;
- export report;
- preview image generation.

**Gate:** reference assets pass automated validation and open correctly in at least the defined interoperability test applications/workflows.

## 20.6 Phase 5 - Desktop Alpha for Windows/Linux

Deliver:

- signed/packaged installers when release process is ready;
- recent projects;
- native file dialogs;
- drag/drop project opening;
- configurable export folders;
- recovery mode;
- application settings;

- updater integration after signing/release infrastructure exists;
- performance profiling on representative hardware.

**Gate:** a non-developer can install, create, save, reopen and export without a terminal.

20.7 Phase 6 - SaaS foundation

Deliver:

- account/login;
- cloud project list;
- upload/download/version sync;
- storage quotas;
- browser editor deployment;
- local/cloud project conflict policy;
- billing abstraction and plan entitlements;
- telemetry and audit events needed for operations.

**Gate:** the same project opens in desktop and browser using the same project schema.

20.8 Phase 7 - AI, marketplace and collaboration

Deliver incrementally:

- prompt-to-generator-parameters;
- AI provider adapters;
- cloud generation credits;
- asset/template marketplace;
- public/private asset libraries;
- team workspaces;
- sharing and review links;
- later real-time collaboration if justified.

21 Suggested planning horizon

This is a prioritization map, not a fixed contractual schedule.

| Horizon    | Primary objective             | Visible outcome                                         | Business meaning |
|------------|-------------------------------|---------------------------------------------------------|------------------|
| Immediate  | Foundation + vertical slice   | Cube/object dashboard; editable in save/open/export GLB | Technical proof  |
| Short term | Core editor + first generator | User creates several asset variants without code        | Usable alpha     |

| Horizon      | Primary objective          | Visible outcome                                  | Business meaning         |
|--------------|----------------------------|--------------------------------------------------|--------------------------|
| Short-medium | Export + desktop packaging | Windows/Linux product usable by external testers | First sellable candidate |
| Medium       | More generators + polish   | Asset creation library and presets               | Product differentiation  |
| Medium-long  | SaaS accounts/cloud        | Same editor in browser with cloud projects       | Subscription foundation  |
| Long         | AI + marketplace + teams   | Creation ecosystem rather than single editor     | Platform business        |

## 22 First implementation backlog

The programmer can start with this ordered backlog.

1. Create monorepo and strict package boundaries.
2. Create `domain` and `application` packages with minimal project model.
3. Create shared editor shell with empty top/left/right/bottom regions.
4. Mount Three.js viewport adapter in center region.
5. Add camera orbit/pan/zoom and grid.
6. Add one cube from domain description.
7. Add picking/selection.
8. Add move/rotate/scale transform tool through commands.
9. Add inspector transform fields synchronized with viewport.
10. Add Undo/Redo command stack.
11. Add color/material edit.
12. Add New/Save/Open project using browser storage first.
13. Add GLB export adapter using Three.js GLTFExporter.
14. Add Khronos glTF validation step.
15. Wrap same editor in Tauri desktop host.
16. Add Tauri filesystem save/open adapter.
17. Add first procedural `SciFiCrateGenerator`.
18. Add generator parameter schema and Randomize/Seed controls.
19. Add automated end-to-end test: create -> save -> reopen -> export -> validate.

## 23 Acceptance criteria for the first public-looking demo

The demo should be considered acceptable only when all are true:

Application starts on Windows and Linux development targets.

Main workbench has toolbar, scene tree, viewport, inspector and status area.



User can create at least Box and Sci-Fi Crate.  
Object can be selected with mouse.  
Move/rotate/scale works visually and numerically.  
Color/material properties update live.  
Undo and Redo cover every supported edit.  
Save/Open preserves the asset exactly enough for continued editing.  
Export GLB works from the UI.  
Validator report is shown in the dashboard.  
Exported GLB can be reopened by the platform itself.  
Generator seed is visible and repeatable.  
No feature module directly imports another module's internals.  
Test suite and CI are green.

## 24 What not to build yet

To keep visible progress high, postpone these until the core product is proven:

- real-time multiplayer collaboration;
- microservice decomposition;
- Kubernetes;
- advanced sculpting;
- full UV editor;
- rigging system;
- animation timeline;
- internal payment marketplace before export quality is proven;
- multiple AI providers before one provider contract is validated;
- proprietary binary project format before the schema stabilizes.

## 25 Definition of architectural success

The architecture is working correctly when a programmer can perform each of the following without touching unrelated modules:

- add a new generator;
- add a new exporter;
- replace local persistence with cloud persistence;
- add a new inspector panel;
- disable AI entirely;
- package the same editor as desktop;
- deploy the same editor in browser;

- migrate a project schema version;
- run domain tests without a browser or WebGL;
- run a 3D renderer test without booting the SaaS backend.

If adding one exporter requires edits throughout the scene, dashboard, persistence and generator code, the modular boundary has failed.

## 26 Final architecture mandate for the programmer

**Build the product as a shared-core hybrid platform.** The first user-facing release is a Windows/Linux desktop editor because it offers the shortest path to a useful professional creation tool. However, the UI and domain/application core are browser-compatible from day zero, so the same editor becomes the SaaS frontend later. Use Clean Architecture as the dependency rule, a modular-monolith product structure, explicit ports/adapters, commands for edits, domain events for dashboard updates, and registries for generators/exporters/plugins. Three.js is an infrastructure/render adapter, not the business model. GLB/glTF is the primary interchange/export path and each export is validated before being reported as successful.

### Immediate target

The next engineering target is intentionally narrow: **one workbench, one cube, one procedural crate, full transform interaction, save/open, undo/redo, GLB export and automated validation**. That vertical slice proves the architecture while producing something visible and useful immediately.

## References and technical basis

- R1. Tauri v2, "Architecture." <https://v2.tauri.app/concept/architecture/>
- R2. Tauri v2, "What is Tauri?" <https://v2.tauri.app/start/>
- R3. Tauri v2, "Webview Versions." <https://v2.tauri.app/reference/webview-versions/>
- R4. Three.js, "GLTFExporter." <https://threejs.org/docs/pages/GLTFExporter.html>
- R5. Three.js, "TransformControls." <https://threejs.org/docs/pages/TransformControls.html>
- R6. Three.js, "OBJExporter." <https://threejs.org/docs/pages/OBJExporter.html>
- R7. Khronos Group, "glTF - Runtime 3D Asset Delivery." <https://www.khronos.org/glTF/>
- R8. Khronos Group, "glTF 2.0 Specification." <https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html>
- R9. Khronos Group, "glTF Validator." <https://github.com/KhronosGroup/glTF-Validator>
- R10. React Three Fiber, "Introduction." <https://r3f.docs.pmnd.rs/>
- R11. Blender Manual, "glTF 2.0." [https://docs.blender.org/manual/en/4.2/addons/import\\_export/scene\\_gltf2.html](https://docs.blender.org/manual/en/4.2/addons/import_export/scene_gltf2.html)
- R12. Unity, "glTFast Documentation." <https://docs.unity3d.com/Packages/com.unity.cloud.gltfast@latest/>
- R13. Epic Games, "Importing glTF Files Into Unreal Engine." <https://dev.epicgames.com/documentation/unreal-engine/importing-gltf-files-into-unreal-engine>
- R14. Godot Engine, "Importing 3D Scenes." [https://docs.godotengine.org/en/stable/tutorials/assets\\_pipeline/importing\\_3d\\_scenes/index.html](https://docs.godotengine.org/en/stable/tutorials/assets_pipeline/importing_3d_scenes/index.html)
- R15. Tauri v2, "File System Plugin." <https://v2.tauri.app/plugin/file-system/>
- R16. Tauri v2, "Capabilities." <https://v2.tauri.app/security/capabilities/>
- R17. Tauri v2, "Updater." <https://v2.tauri.app/plugin/updater/>

This document intentionally separates product decisions from implementation details so the application can evolve without rewriting the core architecture.